Festival Shift Roster Optimizer

A constraint model, three solvers and sick-leave repair for college fest rostering

Gaurav Choudhury, Ansh Sharma, Harshil Chukkala, Ishan Alam, Mayookh Ghosh, Pranay Bansal, Soumya Pathak, Ishan Kumar, Vineet Arora, Yashashwani Dixit

November 2026

Abstract

A college fest rosters its volunteers on a spreadsheet that cannot enforce rest gaps, weekly hour caps or consecutive-day limits, cannot level work between students of the same group, and cannot absorb a dropout without a rewrite. We formulated the roster as a constraint optimisation problem over four independent groups, with two hard rules, four priced rule breaches and three soft terms, and solved it with a greedy constructor, simulated annealing and CP-SAT, plus a local and a CP-SAT repair pass for sick leave.

One evaluator rescored every output. CP-SAT produced the lowest cost on all five curated cases and proved optimality on two of them. Over 100 generated cases local search took 52.3% of the wins, and across 400 solver runs not one run broke a hard rule. Repair changed 1.68 assignments on average with local repair and 2.08 with CP-SAT.

Measure	Value
local search win rate	52.3%
hard-rule failures in 400 runs	0
changes per repair (local / CP-SAT)	1.68 / 2.08

1 Introduction

A college fest rosters its volunteers with the four student groups, MPR, CND, Hospitality and Spons (SPONS in the case files), sharing one spreadsheet. Three failures recurred. Compliance errors went unnoticed: rest gaps, weekly hour caps and consecutive-day rules were broken without anyone seeing it. Workloads drifted apart between students of the same group. And when someone dropped out, a manual fix rippled through the whole sheet.

We built an engine that produces a conflict-free schedule, reports every compromise it had to make, and repairs itself after a volunteer is sick without reshuffling everyone. Assigning students to shifts under coverage demand, rest and workload rules is a small variant of the nurse rostering problem that Burke et al. (2004) survey; our instance has a fixed horizon, four groups that share no students, and preferences rather than contracted hours.

The engine is written in Python and laid out as a loader, a rules module, three solvers, a repair pass and a comparison harness.

2 Problem formulation

A case holds a set of students S, a set of days D, a set of shifts H and a set of groups G, with every student in exactly one group. Demand $r_{d,h,g}$ is the number of students of group g needed on shift h of day d. The binary variable $x_{s,d,h}$ is 1 when student s works shift h on day d, and the slack $u_{d,h,g} \geq 0$ is the shortfall in one demand cell:

$$\sum_{s \in S_g} x_{s,d,h} + u_{d,h,g} = r_{d,h,g}, \quad u_{d,h,g} \geq 0$$

No student works more than one shift a day:

$$\sum_{h \in H} x_{s,d,h} \leq 1 \qquad \forall s \in S, d \in D$$

and a student covers only demand of their own group, so $x_{s,d,h}$ exists only for the group s belongs to. The objective is the weighted sum of the terms in Table 2: the unfilled slack at 3 per missing person, the other three priced rule breaches (C_{over}, C_{streak}, C_{rest}) and the three soft terms (C_{pref}, C_{fair}, C_{iso}), each defined in the next section.

$$\min \quad \sum_{d \in D} \sum_{h \in H} \sum_{g \in G} 3u_{d,h,g} + C_{\text{over}} + C_{\text{streak}} + C_{\text{rest}} + C_{\text{pref}} + C_{\text{fair}} + C_{\text{iso}}$$

A demand cell belongs to one group, and a student covers only their own group, so the model splits into four independent subproblems, one per group.

Each case folder holds five CSV files, listed in Table 1. A preference is one of three types, an off day, a shift the student prefers or a shift to avoid, each with a weight, and a demand grid is keyed by day, shift and group.

Table 1: The five files of a case folder and their columns.

File	Columns
`students.csv`	roll_no, name, group, max_weekly_hours
`shifts.csv`	shift_id, name, start, end, hours, crosses_midnight
`demand.csv`	day, shift_id, group, required
`preferences.csv`	roll_no, type, day, shift_id, weight
`config.csv`	key, value, description

3 Cost terms

Every row of every benchmark is scored by one evaluator, never by a solver's own objective. Two rules are hard and never priced: one shift per day, and own group only. Everything else is a breach with a price, listed in Table 2.

Table 2: The seven priced terms of the evaluator, with their default values.

Term	What it charges
Unfilled slot	3 for a missing person in a cell
Overwork	2, 4 or 6 per extra shift in every rolling window
Streak	4 for a worked day beyond the consecutive-day limit
Rest	2 for a short gap, plus 1 for each further short gap in the same run
Preference	0.05 times the share of preference weight left unmet
Fairness	0.05 times the mean group spread divided by the horizon times the longest shift
Isolated day off	0.05 times the work-off-work count divided by the students times the horizon minus two

The three soft terms each stay below 1 unit, so the soft part of the cost stays below a single rule breach: one unfilled slot costs 3 and one overworked shift at least 2. The seven terms are reported separately for every run, alongside the number of unfilled slots, the preference satisfaction, the per-group workload spread and the runtime.

The overwork charge is computed per student and per rolling window. With W a window of 7 consecutive days and $m_s = \lfloor \text{cap}_s / \text{ shift hours} \rfloor$ the shifts the weekly cap allows, the extra shifts in the window are

$$e_{s,W} = \max\left(0, \sum_{d \in W} \sum_{h \in H} x_{s,d,h} - m_s\right)$$

and the 1st extra shift in a window costs 2, the 2nd costs 4 and each further one costs 6, so the running totals for one, two, three and four extra shifts are 2, 6, 12 and 18. Every window is priced by the same code in the evaluator and in each solver.

The three soft charges carry a factor of 0.05 and normalise against the instance:

$$C_{\text{pref}} = 0.05 \cdot \frac{w_{\text{unmet}}}{w_{\text{total}}}, \quad C_{\text{fair}} = 0.05 \cdot \frac{|\sigma|_g}{|D| \cdot h_{\max}}, \quad C_{\text{iso}} = 0.05 \cdot \frac{n_{\text{iso}}}{|S| \cdot (|D| - 2)}$$

where w_{unmet} is the preference weight left unmet out of the total weight w_{total}, $|\sigma|_g$ is the mean workload spread across the four groups on a scale whose unit is the horizon $|D|$ times the longest shift $h_{\max}$, and n_{iso} counts the work-off-work patterns across $|S|$ students in a horizon of $|D|$ days.

The defaults set a weekly cap of 20 hours, a rolling window of 7 days, at most 5 consecutive days and an 11-hour rest gap between shifts. A case's `config.csv` can override each one; unknown keys, non-numeric values and fractional integer fields raise an error instead of being ignored. The CP-SAT proven gap is the only solver-reported number and is kept apart from the evaluator's cost.

4 Methods

The **greedy** solver fills one person at a time in a hard-first slot order: highest required-to-group-size ratio, then peak day, then Night shifts, with a seeded tie-break. Candidates must fit every affected rolling window under an enabled hard ceiling, and then are ranked by marginal overwork, streak and rest cost using the evaluator's own pricing. A slot is left unfilled only when the cheapest candidate costs more than the unfilled penalty, and rosters are never overstaffed.

Local search starts from the greedy roster and runs simulated annealing (Kirkpatrick, Gelatt and Vecchi, 1983) over reassign, swap, fill, drop and shift-change moves. Its starting temperature is calibrated on sampled worsening moves to accept about 30% of them, cooling is geometric, and there is one reheat after a long stagnation; the best roster seen is always returned. It keeps per-student worked days, rule costs, preferences and per-cell coverage, and a move's delta recomputes only the affected students, days, cells and group.

The **CP-SAT** model, built with OR-Tools (Google), uses the fact that groups never share students: each of the four groups is an independent subproblem, and the four are solved in parallel with the time limit split between them. The model prices every term exactly as the evaluator charges it, and its objective is that group's share of the reported cost. The warm start passes the greedy roster as a hint; the cold start uses the same time and worker split with no hint.

Repair is a separate pass. Five deterministic sick events per case are repaired twice, once by a local method that looks for a free same-group replacement or a chain of at most two changes, and once by CP-SAT over the affected days. Every solve writes its roster and a report with the component breakdown that Table 2 lists.

5 Experimental setup

Three benchmarks ran with seed 1, and every row of each was scored by the same evaluator.

The curated comparison ran greedy, local search, CP-SAT cold and CP-SAT warm on five case folders with a 60 s limit per solver: a balanced 20-day fest, a tight 28-day one, an overloaded 20-day one, a small 7-day one, and one generated tight 28-day case.

The stress benchmark drew 100 cases with a fixed scenario mix: 53 balanced, 27 tight and 20 overloaded, horizons of 7, 14, 20 and 28 days, group sizes from 6 to 30 students and preference fractions from 0.2 to 0.9. It ran the four configurations per case with the same 60 s limit and one search worker per group.

Repair ran on the same 100 cases, with five deterministic sick events per case and a 5 s limit, 500 runs per method.

6 Results on the curated cases

Table 3 gives the total evaluator cost of the four configurations on the five curated cases, seed 1 and 60 s per solver.

Table 3: Total evaluator cost per solver, seed 1, 60 s per solver per case. Lower is better.

Case	Greedy	Local	CP-SAT cold	CP-SAT warm
case_01_balanced_20d	2.0311	0.0366	0.0050	0.0050
case_02_tight_28d	299.0456	127.0555	62.0145	62.0165
case_03_overloaded_20d	571.0477	427.0491	351.0227	350.0202
case_04_small_7d	0.0287	0.0113	0.0089	0.0089
case_tight_28d_seed7	338.0462	156.0488	103.0198	99.0222

CP-SAT produced the lowest cost on all five cases, and local search did not come within 5% of it on any of them. On the balanced case the two CP-SAT runs tied at 0.0050 against local search's 0.0366 and greedy's 2.0311; on the small case they tied at 0.0089 against 0.0113 and 0.0287.

The CP-SAT runs reached proven optimality on the balanced 20-day and small 7-day cases and stopped as feasible elsewhere, as Table 4 shows; on both, every group finished OPTIMAL with a gap of 0.0000 and the run ended there.

Table 4: CP-SAT cold status and proven gap per case, seed 1 and 60 s per solver. The gap is the solver's own bound relative to its own objective.

Case	CP-SAT status (cold)	Proven gap
case_01_balanced_20d	OPTIMAL, all four groups	0.0000 (0.00%)
case_02_tight_28d	FEASIBLE, all four groups	0.0063 (0.01%)
case_03_overloaded_20d	FEASIBLE, all four groups	4.0096 (1.14%)
case_04_small_7d	OPTIMAL, all four groups	0.0000 (0.00%)
case_tight_28d_seed7	FEASIBLE, all four groups	20.0095 (19.42%)

The unfilled count is where the compromises show. Greedy passed the hard rules on every case and still left 59 slots unfilled on the tight case and 145 on the overloaded one. A hard-rule pass is not the same as a filled sheet. CP-SAT cut those to 18 on the tight case and 111 on the overloaded one. Preference satisfaction splits away from cost on the balanced case: CP-SAT satisfied 100.0% of the preference weight there while local search reached 67.3%.

The warm start is not free. On the tight case CP-SAT warm finished 0.0020 cost units above cold, and the whole difference sits in the preference term, 96.7% satisfied against 99.2%, with both runs still feasible at the limit. We attribute that gap to the greedy hint, which can steer the multi-threaded search onto a slightly worse near-optimal incumbent; the gap is neither a correctness nor a hard-rule problem.

Table 5 takes the tight 28-day case and lines up the components. Greedy's cost is mostly 177 units for 59 unfilled slots plus 94 for overwork; local search cut the overwork to 46 and the rest penalty to zero; CP-SAT cut the overwork to 8 and satisfied 99.2% of the preference weight against greedy's 51.6%.

Table 5: Components the evaluator reports for the tight 28-day case, seed 1 and a 60 s limit per solver.

case_02_tight_28d	Greedy	Local	CP-SAT cold
Unfilled slots	59	27	18
Overwork cost	94.00	46.00	8.00
Rest cost	28.00	0.00	0.00
Preference satisfied (%)	51.6	33.6	99.2
Runtime (s)	0.21	60.00	19.36

The runtime column carries the price of the answer. Local search used the whole 60 s limit on this case. CP-SAT cold stopped at 19.36 s with status FEASIBLE and a proven gap of 0.0063. Each group was handed only its share of the time limit and the four groups ran in parallel.

7 Results on generated cases

A case is won by every solver tied at the lowest evaluator cost, with ties sharing the win. Table 6 gives the win rates over the 100 generated cases. The 400 solver runs produced no hard-rule failure and no run missed the 90 s mark.

Table 6: Case wins over 100 generated cases, 53 balanced, 27 tight and 20 overloaded, horizons 7, 14, 20 and 28 days, seed 1 and 60 s per solver. Ties split between the tied solvers.

Solver	Weighted wins	Win rate (%)
greedy	0.00	0.0
local	52.33	52.3
cpsat_cold	23.83	23.8
cpsat_warm	23.83	23.8

Local search took 52.3% of the wins. Table 7 gives the cost gap to the best solver on each case, by scenario, with both the mean and the median.

Table 7: Cost gap to the best solver on each case, by scenario, over 100 generated cases, 53 balanced, 27 tight and 20 overloaded, horizons 7, 14, 20 and 28 days, seed 1 and 60 s per solver.

Scenario	Solver	Cases	Mean gap	Median gap
balanced	greedy	53	11.215	8.026
balanced	local	53	0.012	0.001
balanced	cpsat_cold	53	15.158	2.001
balanced	cpsat_warm	53	5.125	0.003
tight	greedy	27	135.596	140.024
tight	local	27	0.972	0.000
tight	cpsat_cold	27	27.872	11.969
tight	cpsat_warm	27	49.244	33.973
overloaded	greedy	20	170.661	177.009
overloaded	local	20	12.363	0.011
overloaded	cpsat_cold	20	9.536	2.982
overloaded	cpsat_warm	20	35.539	20.468

The order holds by horizon as well as by scenario. Local search's mean gap was 0.011 on the 7-day cases and 6.869 on the 28-day ones, while greedy's rose from 11.099 to 140.818 and CP-SAT cold's from 1.080 to 48.024. The 28-day cases are where the mean gaps are widest.

Runtime separates the configurations far more than cost does. Greedy's mean runtime over the 400 runs was 0.339 s, local search spent 50.447 s, and the two CP-SAT configurations used about 21 s of their 60 s limit.

Reliability was level across the four configurations (Table 8): each ran 100 cases, every run passed the hard rules, and no run went past the 90 s mark the harness flags.

Table 8: Reliability of the four configurations over the 100 generated cases, seed 1 and 60 s per solver.

Solver	Runs	Hard-rule failures	Runs over 90 s
greedy	100	0 (0.0%)	0 (0.0%)
local	100	0 (0.0%)	0 (0.0%)
cpsat_cold	100	0 (0.0%)	0 (0.0%)
cpsat_warm	100	0 (0.0%)	0 (0.0%)

7.1 Local search falls behind at full load

The relative gap is not the whole story. Local search first finished more than 10% above the best CP-SAT cost at load 0.847, on case_019_balanced_7d_seed1359585649, and 34 of the 100 cases are more than 10% behind. Below that line the absolute numbers are small: the 27 flagged cases below load 1.0 all had a local cost of at most 0.0446, where a 10% band is a few hundredths of a unit.

The first material violation, with an absolute gap of at least 1.0 cost unit, is at load ratio 1.000 on case_030_tight_14d_seed881875713, where local search cost 24.0489 against CP-SAT's 17.0206. Beyond that line the gap widens with load: on the overloaded 20-day case at load 1.150, case_014_overloaded_20d_seed1561453423, local search cost 375.0426 against CP-SAT's 330.0211. Figure 1 plots that gap against the load ratio for the 34 flagged cases.

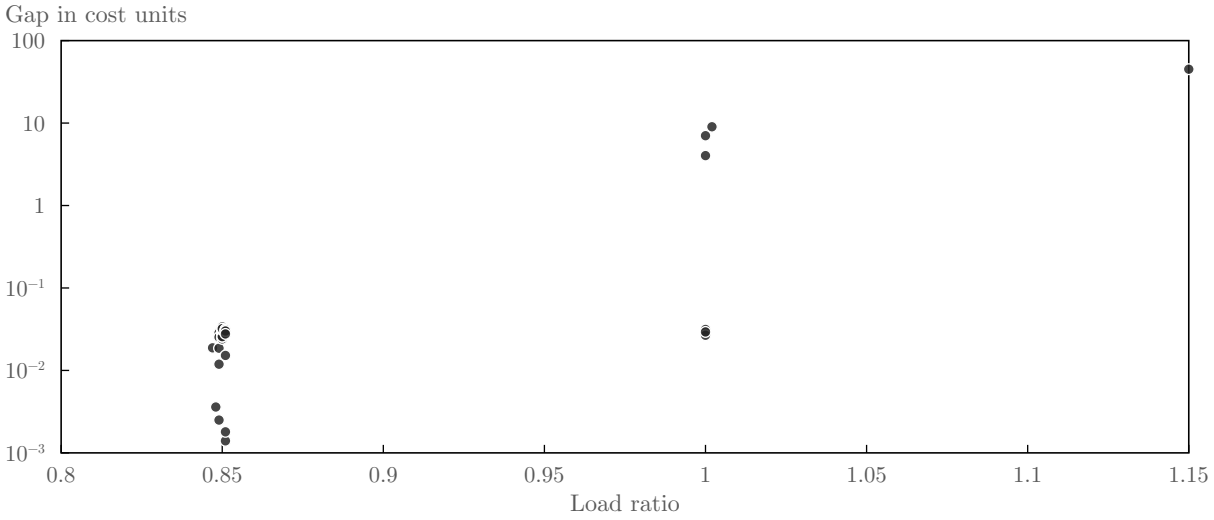


Figure 1: Absolute gap between the local search cost and the best CP-SAT cost against load ratio, over 100 generated cases: 53 balanced, 27 tight, 20 overloaded, horizons 7, 14, 20 and 28 days, seed 1 and 60 s per solver. Shown are the 34 cases where local search finished more than 10% above the best CP-SAT cost; the gap is a few hundredths of a cost unit below load 1.0 and grows to tens of units at load 1.150.

8 Sick-leave repair

After the solve, five deterministic sick events per case are repaired with both methods, and each event is recorded with the number of changed assignments, the runtime and whether the sick slot stayed unfilled. Each summary row in Table 9 covers 500 repair runs. Repair kept the

sheet almost intact: local repair changed 1.68 assignments on average and 2 at the median, CP-SAT 2.08 on average and 2 at the median.

Table 9: Sick-leave repair over 500 events per method on the 100 stress cases, 5 s repair limit.

Method	Runs	Mean changes	Median changes	Median runtime (s)
local	500	1.68	2.00	0.024
cpsat	500	2.08	2.00	0.190

Local repair stops searching when no free same-group replacement or chain of at most two changes beats the unfilled penalty of 3 per slot once the 0.5 per changed assignment is added. Neither method produced a hard-rule failure in 500 repairs each, and both are cheap in wall-clock terms: the median is 0.024 s for local repair and 0.190 s for CP-SAT.

The repair notes say the unfilled flag likely counted the sick slot even when it was already short before the sick event, so it overstates what repair itself left open: 44 of the 306 flagged events were slots nobody had filled before the volunteer fell ill.

9 Limitations

(a) CP-SAT split its time limit across four groups that run in parallel, and this is why it used about 21 s of the 60 s it was given. The ranking on generated cases therefore likely understates CP-SAT against the other solvers. The split is fixed in the code and the stress results need a rerun before the ranking can be read as a verdict.

(b) The stress harness ran CP-SAT with one search worker per group.

(c) All 100 stress cases are generated, and no schedule here was run at a real fest.

(d) The loader rejects demand above group size. An over-subscribed cell cannot be represented as shortage, and allowing it would let the unfilled slack carry that case.

(e) The curated comparison rests on five case folders, enough to rank the solvers there and not enough to generalise from.

10 Conclusion

On the curated cases CP-SAT was the best solver and proved optimality on the balanced 20-day and small 7-day cases. Over 100 generated cases local search was a fast, strong fallback below full load, and it fell behind once the load ratio reached 1.0. Repair needed about two changes per sick event. Across 400 solver runs and 500 repairs per method, no run broke a hard rule.

References

- Burke, E. K., De Causmaecker, P., Vanden Berghe, G. and Van Landeghem, H. (2004). The state of the art of nurse rostering. *Journal of Scheduling*, 7(6), 441-499.
- Kirkpatrick, S., Gelatt, C. D. and Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680.
- Google. OR-Tools CP-SAT solver documentation. `https://developers.google.com/optimization/cp/cp_solver`

Appendix

The repository, `github.com/qwertyduh/CS2105-group-assignment`, holds the whole pipeline. `python3 -m roster.compare --time 60 --seed 1` writes `results/compare.csv`

and `results/compare.md`, and `python3 -m roster.stress run --n 100 --time 60 --workers 4 --repair-time 5 --seed 1` writes `results/stress/manifest.csv` plus one folder per case, which `python3 -m roster.stress report` then summarises in `results/stress/summary.md`. Both use seed 1, cases are generated with `python3 generate_case.py --scenario tight --days 28 --seed 7`, and `python3 -m unittest discover -s tests -v` passes 120 tests. `python3 -m roster.case_io <case>` validates a case folder and prints its load ratio, and `python3 -m roster.rules` prints every rule with its default. Stress case folders and their rosters are rebuilt from the manifest on demand. Table 10 lists what each file holds.

Table 10: Result files under `results/` (stress files under `results/stress/`).

File	What it holds
`compare.csv`, `compare.md`	one row per case and solver: total cost with the seven components, hard pass, preference satisfaction, group spread, runtime and CP-SAT status and gap; the Markdown adds a table per case, the read-out and each ordering warning
`<case>_<solver>.roster.csv`, `.report.json`	one run's assignments, one row per filled cell, and its component breakdown with the hard-rule verdict
`cases/<case>/`, `manifest.csv`	the five case CSVs with `_summary.json`, and the parameters sampled for each stress case
`rosters/`, `summary.md`	the roster every stress run wrote for its case, and win rates, cost gaps, reliability, runtime and the repair table